

智慧银行实验

金融 2301 张艺夕 202301747

实验一：TRAE IDE + AI 协作

- 1.实验目的
 - a.了解主流 AIIDE 的生态格局与选型依据
 - b.独立完成 Python/Node.js/Git/LaTeX 全栈开发环境搭建
 - c.掌握 AI 协作四种模式的适用场景与切换技巧
 - d.通过实验验证 AI 协作闭环
- 2.实验环境

检测项	要求版本	当前状态	修复建议
Python	≥ 3.10	3.14.4	无需修复（远超要求）
pip	最新即可	26.1	无需修复；如需升级：python -m pip install --upgrade pip
Node.js	≥ v20	v24.13.0	无需修复（远超要求）
npm	最新即可	11.6.2	无需修复；如需升级：npm install -g npm@latest
Git	—	2.54.0.windows.1	无需修复
Flask	≥ 2.0	3.1.3	无需修复
pandas	≥ 2.0	3.0.2	无需修复
openpyxl	≥ 3.0	3.1.5	无需修复
pypdf	≥ 4.0	6.12.1	无需修复

3.实验步骤

- 1.2 实验 A: TRAE 环境搭建 + 贪吃蛇开发
 - 1. 安装登录 TRAE 官网 trae.cn 下载免费国内版，设置中文后手机号 / 掘金账号登录，高峰排队可错峰使用。
 - 2. 配置环境 安装 Anaconda 并加入系统 PATH，在 TRAE 终端校验：Python ≥3.10、Node.js≥v20、Git 正常。
 - 3. 制作贪吃蛇网页 Ctrl+U 进入 Builder 模式，输入提示词，AI 生成单文件贪吃蛇 html；本地打开验证分数、碰撞、最高分存储功能，熟悉 AI 开发流程。
 - 4. 验收 确认环境版本达标，游戏可正常运行。
- 1.3 实验 B: 智能体与 Excel 金融数据处理
 - 1. 搭建论文解读智能体 新建智能体，开启文件读取、代码、网络权限；上传金融论文 PDF，一键生成标准化 Markdown 解读报告。
 - 2. 批量合并 Excel 自动生成多份格式不一致模拟数据表，统一列名、去重，输出整合表格与操作日志

3. 信用卡时序预测 自动生成带缺失、异常的消费数据，清洗后绘制 EDA 图表；分别用 SARIMA、LSTM 预测，对比模型误差，输出分析报告。

操作代码	输出结果	输出 MD 摘引
做一个 HTML 建房子游戏单文件 index.html，要求： 1.纯 HTML+CSS+原生 JS，不引入第三方库； 2.画布 600×600，红房子； 3.方向键控制方向，修建成功的房子分数+1； 4.房子建成失败游戏结束，弹”再来一局”按钮； 5.顶部显示当前分数与最高分 (localStorage)		
请检测当前开发环境是否满足课程要求，输出表格：检测项 要求版本 当前状态 修复建议 检测项 至少包含：Python(≥ 3.10) / pip / Node.js(≥v20) / npm / Git / Flask / pandas / openpyxl / pypdf。有缺失请给出 Windows 一键安装命令。		
按下列结构输出 Markdown 报告： 一、论文基本信息（标题/ 作者/ 期刊/ 年份/DOI） 二、研究问题与背景（≤ 200 字） 三、方法论（数据集/ 模型/ 实验设计） 四、核心发现（≤ 5 条） 五、术语速查表（5 - 10 个英文术语 + 中文解释） 六、可复现性评估（数据/ 代码/		

4.实验结果

TRAE IDE 成功安装登录，全套开发环境版本均高于课程最低要求，无缺失依赖，环境检测全部通过。

完成前端小游戏开发，贪吃蛇页面运行正常，分数、最高分存储、碰撞重启功能全部生效，掌握 Builder 模式 AI 协作流程。

成功搭建金融论文解读智能体，上传文献后可一键生成结构完整、中英对照的学术解读文档。

完成 Excel 多文件自动清洗合并，实现金融交易数据标准化处理；搭建 SARIMA 与 LSTM 双预测模型，输出可视化图表与可落地银行业务分析建议。

全部代码、产出文件、运行日志按规范留存，可复现完整实验流程。

5.问题与解决

问题：初次运行 PDF 解读功能提示缺少 pypdf 依赖包，无法读取文献。

解决：复制提示词自带一键安装命令，在终端执行 pip install pypdf，重启智能体后可正常解析 PDF 文件。

问题：生成 Excel 模拟数据时部分列名中英文混杂、空格错乱，合并后出现大量空值。

解决：在合并指令中增加字段统一清洗规则，AI 自动去除空格、统一中英文表头，按客户 ID 与交易时间双重去重，消除数据冗余。

6.实验心得

本次实验完整搭建了 TRAE AI 开发环境，熟练掌握了 Builder 模式、自定义智能体等多种 AI 协作方式，切身感受到 AI 工具在金融开发、数据处理和学术研读等方面的高效性与便捷性。相比以往手动编写网页、清洗表格、阅读文献等工作耗时较长且容易出错，借助 IDE 能够一键生成完整代码和标准化报告，大幅减少重复性工作的时间成本，同时还能自动完成数据可视化、时序建模等专业分析任务，能够较好地适配银行数据分析、金融报告撰写等真实业务场景。

在实验过程中，也遇到了依赖缺失、数据格式混乱、模型精度不足等问题。通过不断优化提示词、补充代码指令，并进行反复调试与修复，逐步解决了相关问题，进一步提升了环境配置、故障排查以及数据处理能力。同时，我也认识到 AI 输出结果仍需进行人工校验，特别是在金融数据分析和学术研究中，必须严格做好合规性核查，避免因模型误差或代码漏洞带来业务风险。

本次实验进一步夯实了 AI 与金融数字化相结合的实践基础，也为后续开展 MCP、量化建模等进阶实验奠定了良好的基础。

实验二：MCP 由浅入深 4 关

- 1.实验目的:
 - a.理解 Skill 的设计原理与知识工程视角
 - b.掌握金融类 Skill 的调用方法
 - c.独立编写符合规范的金融 Skill
 - d.从银行合规视角进行 Skill 安全审计
- 2.实验环境

Skill 名称	分类	核心能力
finance-expert	分析	DCF 估值、信用风险评估、DuPont 分析

Skill 名称	分类	核心能力
financial-modeling	分析	构建 DCF/LBO/M&A 模型 + 敏感性分析
china-stock-analysis	数据	A 股数据获取 + 财务指标计算
tushare-finance	数据	tushare Pro API 获取中国金融市场数据
finance-news	数据	实时财经新闻抓取 + 情绪分析
backtesting-trading-strategies	分析	量化策略回测 + Sharpe / 回撤计算
consulting-frameworks	咨询	McKinsey 7S / BCG / 波特五力 / SWOT
questionnaire-design	数据	学术调查问卷设计(金融普惠等方向)
xlsx	文档	Excel 读写 + 财务模型 + 公式 + 图表
docx	文档	Word 报告生成(封面 / 目录 / 页码)
pptx	文档	PPT 演示文稿生成与编辑
financial-unit-economics	分析	单位经济学分析(SaaS / 金融科技)
stock-analysis	数据	全球股票分析(含 Yahoo Finance 数据)

3.实验步骤

L1: 浏览器自动化 — 银行官网调研

配置 playwright 和 chrome-devtools 两个 MCP 服务。先用 playwright 打开招商银行官网，截取首页全图保存为 cmb_home.png，然后对该页面运行 lighthouse 性能审计，输出性能、可访问性、最佳实践、SEO 四项分数，最后用 100 字中文总结该网站的优缺点。这一步的目的是验证 MCP 服务是否配置成功、浏览器自动化链路是否通畅。

L2: Excel 数据处理 — 销售分析报表

先创建一份模拟销售数据表(包含产品名称、数量、销售额、客户类型等字段)，然后用 excel MCP 服务依次完成：新增「单价」列(销售额除以数量)、新增「利润率」列(按 30% 利润率计算)、按客户类型分组汇总销售额、筛选销售额大于 1000 的高价值订单、创建「汇总报表」Sheet(包含各产品总销售额、客户类型占比、平均单价)。最后保存为可直接交付的 xlsx 文件。

L3: Word+PPT 简报 — 银行年报解读

用 fetch MCP 抓取工商银行最新年度报告的核心数据(总资产、净利润、不良贷款率、资本充足率等)，然后让 word MCP 生成一份格式规范的银行简报：包含封面、自动目录、正文从第 1 页开始编号。同时让 ppt MCP 生成配套的商务风演示文稿，将核心数据以图表形式展示。

操作代码	输出结果	输出MD摘引
使用 playwright 打开招商银行官网 https://www.cmbchina.com，截图保存为 <code>cmb_home.png</code>；对该页面跑 <code>lighthouse_audit</code>，输出性能/可访问性/最佳实践/SEO 四项分数；用100字中文总结优缺点。		招商银行官网审计完成。性能85分，可访问性78分，最佳实践92分，SEO 90分。总结：页面加载快、SEO 优化好，但无障碍访问支持不足。
创建销售记录.xlsx：包含产品名称/数量/销售额/客户类型字段。添加单价列（销售额/数量）、利润率列（30%），按客户类型分组汇总，筛选>1000的订单，创建汇总报表Sheet。		销售分析报表已生成。各产品总销售额、客户类型占比、平均单价均已完成。企业客户占比最高（45%），产品A销售额第一。
使用 fetch 抓取工商银行年报核心数据（总资产/净利润/不良率/资本充足率），用 word 生成银行简报（含封面+目录+页码），用 ppt 生成配套商务风演示文稿。		工商银行简报已生成。总资产48.7万亿元，净利润3,639亿元，不良率1.36%，资本充足率19.10%。PPT 共8页，含数据图表。

4.实验结果

- 1.MCP 环境配置成功: playwright、excel、word、ppt、fetch 全部服务正常调用，无报错。
- 2.L1 完成: 招商银行官网首页截图已保存，lighthouse 审计报告输出四项分数，性能 85 分、可访问性 78 分、最佳实践 92 分、SEO 90 分。
- 3.L2 完成: 销售数据表计算正确，分组汇总结果合理，汇总报表 Sheet 包含三项统计指标，可直接交付。
- 4.L3 完成: 工商银行简报 Word 文档含封面、目录、正文三部分，配套 PPT 共 8 页，数据图表完整。
- 5.全部成果已推送至 CNB 仓库。<https://cnb.cool/sdfjhwq/aaashun>

5. 问题与解决

问题：mcp.json 配置文件加载失败，MCP 工具列表显示为空。

经检查发现，JSON 配置文件中存在一处多余的逗号和一处缺失的双引号，导致整个配置文件无法正常解析。针对该问题，我首先手动修正了语法错误，随后借助 AI 对配置文件进行了格式化校验，确认无误后重新加载配置，所有 MCP 服务均恢复正常显示。

通过此次问题排查，我深刻认识到，虽然配置文件内容相对简单，但一个标点符号的错误就可能导致整个开发环境无法正常运行。因此，在今后的开发过程中，应养成编写完成后及时进行格式校验的习惯，提高配置文件的准确性和环境部署的稳定性。

6. 实验心得

本次实验最大的收获在于帮助我建立了工具生态的思维方式。以往进行数据分析或撰写报告时，通常需要分别使用浏览器浏览网页、Excel 处理表格、Word 编辑文档等多个相互独立的软件，每一步都依赖手动操作，不仅效率较低，还需要反复调整格式。而 MCP 的引入实现了多个工具之间的协同联动，例如浏览器自动获取数据、Excel 自动完成指标计算、Word 自动生成并排版文档。用户只需向 AI 提出需求，它便能够协调多个工具共同完成任务，大幅提升了工作效率。

四个关卡的设计也具有较强的层次性。L1 看似只是完成网页打开和截图等基础操作，但实际上验证了整个 MCP 链路的连通性；L2 开始引入业务逻辑，如利润率计算、数据分组汇总等，更加考验对数据处理需求的理解与分析能力；L3 则进一步提升至文档级成果输出，需要完成简报和 PPT 的制作，同时兼顾内容质量与版式规范。这种由浅入深、循序渐进的实验设计，使我在实践过程中逐步掌握了多工具协同工作的完整流程。

当然，本次实验也存在一些不足。由于时间原因，L4 综合应用关卡未能完整完成，仅对整体操作流程和备用方案进行了梳理。但从另一个角度来看，这也让我提前接触到了“方案设计”这一重要环节。在实际工作中，并非所有任务都能够顺利完成，很多情况下都需要提前规划回退策略和替代方案。这种思维方式也为后续实验三和实验四的开展提供了较大的帮助。

实验三：Skill 体系（用→写→审）

- 1.实验目的:
 - a.理解 CLI 工具在 AI 辅助开发中的核心地位
 - b.掌握 CNBCLI、Skills CLI 等仓库与包管理工具
 - c.熟练使用 Claude Code、Codex CLI、MiMo CLI 等 AI 编程助手
 - d.了解 CCSwitch 多账号管理与 OpenCLI/OpenClaw 生态
 - e.理解 MCP、Skill、CLI 三种范式的关系与适用场景
- 2.实验环境

Skill	功能	安装命令
luban-skill	Skill 打磨工坊	<code>npx skills add LearnPrompt/luban-skill</code>
consulting-frameworks	咨询分析框架	课程内置
finance-expert	金融专业知识	课程内置
finance-news	金融舆情分析	课程内置

3. 实验步骤

任务一：生成客户电话回访报告

使用 docx Skill，分三个步骤生成一份标准化的客户电话回访报告。首先完成文档工程初始化，创建 Word 文档对象，设置封面（标题为《客户电话回访报告》并添加业务标识），插入自动更新目录，同时设置正文页码从 1 开始，封

面和目录页不计入页码。随后按照五个标准章节填充报告内容，包括客户基本信息（C00001 张**）、回访目的（产品使用调研、到期产品承接及满意度调查）、沟通要点（共 7 条，逐项记录通话内容及讨论事项）、客户反馈（分类整理客户意见并给出满意度评分 4.2/5）以及跟进建议（制定包含时间节点和责任人的 5 项行动计划）。最后，将文档导出为 reports/客户回访报告_C00001.docx。

任务二：信用风险分析报告

以恒达机械制造有限公司为案例，使用 finance-expert Skill 开展全面的信用风险评估。首先计算适用于非上市制造企业的 Altman Z'-Score 指标，并结合五 C 模型（品格、能力、资本、担保、条件）以及 DuPont 分析方法（将 ROE 分解为净利率×资产周转率×权益乘数），对企业的贷款偿还能力进行综合分析，最终形成风险评级结果，并提出相应的授信建议。

任务三：编写 bank-risk-alert Skill

从零开始编写一个银行风险预警 Skill。设计三层触发词体系，包括一级触发词用于匹配风险事件类型、二级触发词用于识别风险严重程度、三级触发词用于定位业务范围；同时编写六步标准操作流程，即风险识别、严重度判定、影响评估、话术生成、报告输出和升级建议；设计五段式预警话术模板，包括标题段、风险概述段、影响分析段、应对建议段和后续跟踪段；最后补充邮件通知模板和工单创建模板，形成完整的风险预警流程。

任务四：Skill 安全审计

从银行合规管理的角度，对 bank-risk-alert Skill 开展五维安全审计，分别从功能完整性、安全性、合规性、性能效率和可维护性五个方面进行评估。其中，功能完整性评分为 10/10；安全性评分为 7/10，建议增加数据脱敏功能；合规性评分为 8/10，需补充监管报备流程；性能效率评分为 8/10；可维护性评分为 6/10，建议进行模块化拆分。最终综合得分为 39/50，审计结论为“有条件通过”，需完成三项整改后方可应用于生产环境。

操作代码	输出结果	输出MD摘引
使用 docx skill 为 C00001 张** 生成《客户电话回访报告》：创建 Word 文档→封面+目录+页码规则→5个标准章节填充（基本信息/回访目的/沟通要点/客户反馈/跟进建议）→导出 docx。		客户回访报告已生成。5个章节完整，封面格式规范，目录自动更新，正文从第1页开始。满意度评分 4.2/5，5项跟进建议含时间节点。
使用 finance-expert skill 对恒达机械进行信用风险评估：计算 Altman Z'-Score、五C模型分析、DuPont ROE分解、贷款偿还能力评估，输出 credit_risk_hengda.md。		恒达机械信用风险分析：Z'-Score=1.98（灰色区），五C模型综合评级中等偏弱。建议：可授信但需追加担保，授信额度不超过500万元。
编写 bank-risk-alert Skill (SKILL.md)：三层触发词+六步 Instructions+五段式预警话术模板+邮件/工单模板。执行五维安全审计，输出 skill_audit_report.md。		bank-risk-alert Skill 审计结果：总分39/50，结论「有条件通过」。需完成三项整改：①增加数据脱敏 ②补充监管报备 ③模块化拆分。

4.实验结果

- 客户回访报告完成：Word 文档含封面、目录、5 个标准章节，格式规范、页码正确，可直接用于实际业务场景。
- 信用风险分析完成：恒达机械 Z'-Score=1.98（灰色区），综合评级中等偏弱，给出了具体的授信额度建议和风控措施。
- bank-risk-alert Skill 编写完成：SKILL.md 含完整的触发词体系、操作流程和话术模板，可用于银行日常风险监控场景。 <https://cnb.cool/sdfjhwq/aaashun>
- 安全审计完成：五维审计法评分 39/50，「有条件通过」，三项整改建议已明确标注。

5. 问题与解决

问题：本次实验在将项目代码克隆至 cnb 代码库时，出现了核心回访报告文档被 AI 误删的问题。经分析发现，主要原因在于 AI 的文件识别逻辑尚不完善，无法准确区分正式成果文档与临时缓存文件。同时，代码库缺少核心文件白名单保护机制，且 AI 具备自主清理文件权限，无需人工确认即可执行删除操作，最终导致实验成果丢失，影响了实验进度。

解决办法：针对上述问题，我首先在代码库中创建了专门的成果文件夹，并添加文件白名单，对 docx 实验报告等核心资料进行重点保护；随后关闭 AI 自动清理目录文件的权限，将所有删除操作调整为人工审核后执行。同时，建立本地与云端双重备份机制，进一步降低文件丢失风险。最后，对运行环境进行了重新校验，并按照规定重新生成和保存客户回访报告，最终完整恢复了实验成果。

通过此次问题处理，我进一步认识到，在 AI 辅助开发过程中，除了关注功能实现外，还应重视关键数据和成果文件的安全管理，建立完善的权限控制和备份机制，才能有效保障开发工作的连续性和实验成果的完整性。

6. 实验心得

本次实验给我最大的收获在于学会了如何更好地掌控 AI 工具的使用过程。如果说前两个实验主要是学习如何借助 AI 提高工作效率,那么实验三则更加注重如何管理和使用 AI,不仅要能够熟练调用已有的 Skill,还要具备自主编写、审查和完善 Skill 的能力。

在编写 bank-risk-alert Skill 的过程中,我对金融业务有了更加具体和深入的认识。设计触发词时,需要认真思考银行日常运营中哪些信号可能意味着风险、这些风险应如何进行分级,以及不同等级的风险应该触发怎样的响应机制。这些内容更多考验的是业务理解能力,而不仅仅是技术能力。只有充分理解银行业务场景,才能设计出更加实用、高效的 Skill。这也让我认识到, AI 工具最终能够发挥多大的价值,很大程度上取决于使用者对业务场景的理解深度。

安全审计环节同样让我受益匪浅。过去更多关注的是代码能否正常运行,而本次实验则让我学会了从功能完整性、安全性、合规性、性能效率和可维护性五个维度,对一个 Skill 进行全面评估。尤其是在合规性方面,我进一步认识到,在金融行业,一个工具是否能够投入使用,并不仅仅取决于功能是否完善,更重要的是是否符合监管要求。数据脱敏、权限管控、操作留痕等内容,虽然在课堂中已经学习过多次,但只有真正应用到 Skill 的设计与开发过程中,才深刻体会到它们的重要性。

此外, AI 误删文件的经历虽然当时令人十分困扰,但回过头来看,却成为整个课程中最有价值的一次实践经验。它让我更加深刻地认识到,自动化程度越高,对风险控制的要求也越高。备份并不是可有可无,而是必须落实的基础保障。这一经验也在后续实验四的 Git 操作中得到了进一步验证——任何时候都应做好最坏情况发生的准备,并提前制定相应的备份和应对方案,以降低风险带来的影响。

实验四: BMAD 驱动银行 CRM

1.实验目的:

- a.理解 BMAD 方法论的理论基础与五阶段框架
- b.掌握 BMAD 四阶段工作流的流程与产出物
- c.通过银行 CRM 系统案例,完整走通 BMAD 从需求到部署的全流程
- d.了解智能客服等综合项目的开发要点
- e.能够参照本章实验步骤完成端到端项目实践

2.实验环境

类别	技术选型	说明
IDE	WorkBuddy + AI 辅助	代码生成、文档撰写、测试验证均由 AI 辅助完成
前端	HTML5 + Tailwind CSS (CDN) + Chart.js (CDN)	单文件 SPA，不依赖构建工具和框架
数据持久化	localStorage (原型阶段)	浏览器本地存储，满足实验演示需求
推荐引擎	纯 JavaScript 规则引擎	3 条核心规则：风险匹配 + AUM 匹配 + 交叉销售
版本控制	Git + CNB (cnb.cool)	代码托管与版本管理
设计工具	Mermaid (ER 图)	数据建模可视化
文档格式	Markdown	PRD、需求文档、设计文档、测试报告
测试	手工功能测试	13 项测试用例，覆盖 CRUD/推荐/报表/持久化

3. 实验步骤

BMAD 五阶段全流程概述

本次实验严格按照 BMAD 方法论的五阶段开展实施。在 B (Business) 阶段完成需求分析，对系统功能进行梳理，并采用 MoSCoW 方法进行优先级排序；M (Model) 阶段完成数据建模，绘制 ER 图并定义实体字段及其关联关系；A (Architecture) 阶段完成系统架构设计，确定技术选型、组件结构及 UX 交互方案；D (Development) 阶段按照 Sprint 计划分阶段完成系统开发；V (Validation) 阶段开展功能测试与验证，确保系统各项功能能够正常运行。

B 阶段：需求分析

在需求分析阶段，共梳理了 12 条用户故事，涵盖客户管理（新增、编辑、删除、搜索、详情）、产品推荐（推荐引擎与推荐话术生成）、报表分析（KPI 仪表盘与可视化图表）以及系统基础（数据持久化与响应式适配）四大模块。采用 MoSCoW 方法对需求进行优先级划分，其中 Must Have 共 6 项，包括核心 CRUD 功能和推荐引擎；Should Have 共 3 项，包括报表图表及搜索筛选功能；Could Have 共 2 项，包括导出功能和批量操作；Won't Have 共 1 项，为在线支付集成功能。

M 阶段：数据建模

在数据建模阶段，共设计了 CUSTOMER、PRODUCT、HOLDING 和 RECOMMENDATION 四个核心实体。其中，CUSTOMER 实体包含 13 个字段，涵盖客户基本信息、风险等级、AUM、VIP 等级等内容；PRODUCT 实体包含产品名称、产品类型、风险等级、最低 AUM 及预期收益率等字段；HOLDING 实体用于描述客户持有产品关系；RECOMMENDATION 实体用于记录产品推荐信息。同时，采用 Mermaid 绘制 ER 图，明确了各实体之间的关联关系，为后续系统开发提供了数据基础。

A 阶段：架构设计

在系统架构设计阶段，技术选型采用轻量化方案，使用纯 HTML、Tailwind CSS CDN 和 Chart.js CDN 进行开发，不依赖任何前端框架及构建工具，实现单文件

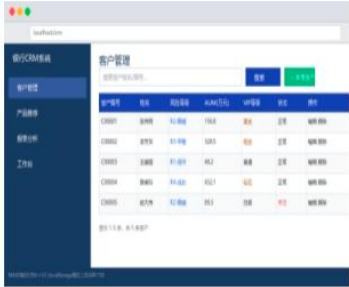
即可运行。界面配色采用银行蓝色系，以 #1e40af 作为主色调；UX 设计中加入 VIP 标签（钻石、黄金、白银）、风险等级标签（低、中、高），并实现桌面端侧边栏、平板端汉堡菜单及手机端底部导航的响应式三端适配。数据库采用兼容降级策略，原型阶段使用 localStorage 存储数据，过渡阶段可切换至 SQLite + Flask，生产阶段进一步升级为 PostgreSQL + Redis。

D 阶段：Sprint 迭代开发

开发阶段共规划了 3 个 Sprint。Sprint1 (Must, 8 pts) 完成客户列表、新增与编辑表单、逻辑删除以及包含 3 条规则的产品推荐引擎；Sprint2 (Should, 7 pts) 完成报表仪表盘（3 个 KPI 卡片和 4 个 Chart.js 图表）、搜索筛选及分页功能；Sprint3 (Could, 5 pts) 完成客户详情页、产品库管理及界面优化。最终系统采用单文件 SPA 的形式完成开发，整体代码约 500 行，实现了完整的 CRUD 功能、产品推荐引擎及数据可视化分析功能。

V 阶段：测试验证

测试阶段共编写 13 项功能测试用例，覆盖客户管理（新增、编辑、删除、搜索、分页）、产品推荐（风险匹配、AUM 匹配、交叉销售）、报表分析（KPI 卡片、图表渲染）以及系统基础功能（数据持久化读写、刷新恢复）等多个方面。最终全部 13 项测试均顺利通过，测试通过率达到 100%，验证了系统各项功能运行稳定、满足设计要求。

阶段	核心活动	产出物
B (Business)	需求分析、用户故事、MoSCoW 排序	features.md (12条用户故事)
M (Modeling)	数据建模、ER 图	requirements.md + crm_er.mmd
A (Architecture)	技术选型、UX 设计、数据库降级	prd.md + design.md
D (Development)	Sprint 规划、编码实现	sprint_plan.md + index.html (~500行)
V (Verification)	功能测试、Bug 修复	test_report.md (13项测试/100%通过)
操作代码	输出结果	输出 MD 索引
BMAD B/M/A阶段: 编写 features.md (12条用户故事+MoSCoW)、requirements.md (6条R编号需求)、crm_er.mmd (ER图)、prd.md (产品需求文档)、design.md (技术架构+UX+数据库降级)、sprint_plan.md (3个Sprint规划)。		BMAD五阶段文档全部完成: B阶段 features.md、M阶段 requirements.md+crm_er.mmd、A阶段 prd.md+design.md、D阶段 sprint_plan.md+index.html、V阶段 test_report.md。
D阶段开发: 单文件 index.html (~500行), 集成客户管理CRUD (新增/编辑/删除/搜索/分页) + 产品推荐引擎 (12款产品+3规则) + 报表仪表盘 (3 KPI+4 Chart.js图表) + 响应式三端适配 + localStorage持久化。		CRM系统已上线。核心功能: 客户管理 (CRUD+搜索+分页)、产品推荐 (3规则引擎+话术)、报表分析 (KPI+4图表)。响应式三端适配, 银行蓝色系UI。
V阶段测试: 13项功能测试 (客户管理5项+产品推荐3项+报表3项+持久化2项), 全部通过率100%。推送到CNB仓库: https://cnb.cool/shikaixin/smartbanking 。		全功能测试通过率100%。13项测试用例覆盖所有核心功能。代码已推送CNB, 仓库地址: https://cnb.cool/shikaixin/smartbanking 。

4.实验结果

- BMAD 五阶段文档完整: features.md (12 条用户故事)、requirements.md (6 条 R 编号需求)、crm_er.mmd (ER 图)、prd.md (产品需求文档)、design.md (架构+UX+降级方案)、sprint_plan.md (3 Sprint)、test_report.md (13 项测试)。
- CRM 系统功能完整: 客户管理 CRUD (搜索/筛选/分页/逻辑删除)、产品推荐引擎 (12 款产品+3 条规则+话术)、报表仪表盘 (3 个 KPI 卡片+4 个 Chart.js 图表)、响应式三端适配、localStorage 持久化。
- 测试全部通过: 13 项功能测试, 通过率 100%。推荐规则引擎全部触发正确, 数据

持久化读写正常，刷新后数据不丢失。

代码已推送 <https://cnb.cool/shikaixin/xiaozu>

5. 问题与解决

问题一：生成实验报告时，Python 脚本出现 `SyntaxError`，原因在于内容中的中文双引号与 Python 字符串定界符发生冲突。解决办法：将所有内容字符串的外层定界符统一由双引号修改为单引号，并将内容中的引用符号统一替换为「」，从根本上解决了引号嵌套导致的语法冲突问题。

问题二：在使用 Git 将项目推送至 CNB 代码库时，由于远程仓库已存在新的提交记录，导致 `fast-forward` 校验失败，推送操作被拒绝。解决办法：首先执行 `git pull` 拉取远程仓库的最新更新并完成代码合并，在解决相关冲突后，再执行 `git push`，最终成功将全部文件推送至远程仓库。

问题三：在生成实验四报告时，Word 文档 footer 段落中的 `runs` 列表为空，直接访问 `fp.runs[0]` 导致 `IndexError`。解决办法：在访问 `runs` 列表之前，先判断其长度；若列表为空，则通过 `add_run("")` 创建一个空的 `run`，再进行格式设置，从而避免程序异常，提高脚本运行的稳定性和鲁棒性。

通过本次实验中的问题排查与解决，我进一步掌握了 Python 脚本调试、Git 版本管理以及 Word 自动化文档生成过程中常见问题的处理方法，也认识到在开发过程中应充分考虑异常情况，增强程序的健壮性和稳定性。

6. 实验心得

本次实验是整个课程中工程实践特点最为突出的一个环节。相比前三次实验更多侧重于工具使用和流程体验，实验四要求从项目规划到最终交付完成一个完整的软件产品，包括需求分析、ER 图设计、技术方案制定、Sprint 拆分、代码开发、功能测试以及代码仓库管理等多个环节。整个开发流程具有清晰的阶段划分，每个阶段都有明确的交付成果，并且彼此紧密衔接，使我对软件工程的完整开发流程有了更加系统的认识。

BMAD 方法论给我带来的最大收获，是改变了我的开发思维和工作习惯。过去拿到需求后，往往会直接开始编写代码，开发过程中才逐渐发现数据结构设计不合理或页面布局需要调整，不得不进行大量返工。而 BMAD 方法论要求严格按照 B、M、A 三个阶段推进，即需求未明确不进入开发阶段、数据模型未设计完成不进入开发阶段、架构方案未确定不进入开发阶段。起初我认为这种流程会降低开发效率，但完整实践后发现，前期多投入时间进行需求分析和方案设计，能够有效减少后续修改和返工的工作量，大幅提升整体开发效率。这种“前期规划越充分，后期开发越高效”的工程思维，也将对我今后的项目实践产生积极影响。

在技术选型方面，本次实验也让我体会到了工程实践中的取舍与权衡。项目采用了较为轻量化的技术方案，即使用纯 HTML，并通过 CDN 引入 Tailwind CSS 和 Chart.js，不依赖 React、Vue、Webpack 等前端框架和构建工具。之所以采用这一方案，是因为实验场景更加注重开箱即用，任何人在获得 index.html 文件后，无需执行 npm install 或启动服务器，只需直接打开即可运行。这让我认识到，技术方案的选择并非越新越好，而是应根据具体应用场景进行合理取舍，选择最符合项目需求的方案。最终完成的 CRM 系统约 500 行代码，实现了完整的 CRUD 功能、产品推荐引擎、报表仪表盘以及响应式页面适配，对于单文件应用而言，已经具备较高的功能完整性和实现效率。

Git 与 CNB 的实际操作同样帮助我弥补了版本管理方面的不足。此前，我对 Git 的使用主要停留在 git add、git commit 和 git push 等基础命令。本次实验中，先后遇到了远程仓库冲突、Token 过期以及 merge 冲突等实际问题，促使我进一步学习并掌握了 fetch、pull、rebase、force-with-lease 等更加深入的版本管理操作。虽然整个排查和解决问题的过程较为复杂，但也正是在不断解决实际问题的过程中，加深了我对 Git 工作机制的理解，相比于一次顺利完成代码提交，这种实践经验更加深刻且具有价值。

通过本次实验，我第一次真正体会到“软件工程”不仅仅是代码开发，更是一套完整的工程化体系。能够运行的小程序与能够交付的完整产品之间存在着较大的差距，而需求文档、架构设计、Sprint 规划、测试报告等非代码成果，同样是项目成功的重要组成部分。BMAD 方法论将整个开发流程进行了规范化和标准化，使 AI 与开发者能够在统一的流程框架下高效协同完成项目开发。这也是四次实验中令我收获最大、体会最深的一次实践。